

HelixCode CLI Reference

Overview

HelixCode provides a comprehensive command-line interface (CLI) for managing all aspects of the enterprise AI development platform. The CLI includes tools for server management, configuration, performance optimization, security scanning, and distributed computing.

Main CLI Commands

helix

The main HelixCode CLI command with comprehensive AI development platform management.

helix [command] [flags]

Global Flags

Flag	Type	Default	Description
--config	string	""	Configuration file path
--debug	bool	false	Enable debug logging
--log-level	string	"info"	Log level (debug, info, warn, error)

Commands

helix start

Start HelixCode with automated local LLM management.

helix start [flags]

Flags: - --auto: Enable full automation (default: true) - --monitor: Enable health monitoring (default: true) - --optimize: Enable performance optimization (default: true) - --check-interval: Health check interval (default: 30s)

Description: Starts HelixCode with fully automated local LLM provider management. Automatically installs, configures, and manages 11+ local LLM providers with zero-touch operation.

Example:

```
# Start with default automation
```

```
helix start
```

```
# Start with custom check interval
```

```
helix start --check-interval=60s
```

```
# Start without optimization
```

```
helix start --optimize=false
```

helix auto

Fully automated local LLM management mode.

```
helix auto
```

Description: Starts HelixCode in fully automated mode where all local LLM providers are automatically cloned, installed, configured, started as background services, and maintained without user intervention.

helix server

Start the HelixCode server.

```
helix server
```

Description: Starts the HelixCode HTTP server with all configured providers and services on the default port (8080).

helix version

Show version information.

```
helix version
```

Output:

```
HelixCode Enterprise AI Development Platform
Version: 1.0.0
Build: 2025.01.20
AI Providers: 29 (18 cloud + 11 local)
Token Context: 2M
License: MIT
```

helix generate

Generate code/text with AI.

```
helix generate [prompt] [flags]
```

Flags: - --model: LLM model to use (default: "llama-3-8b") - --max-tokens: Maximum tokens to generate (default: 1000) - --temperature: Generation temperature (default: 0.7) - --stream: Stream the response (default: false)

Example:

```
# Generate code with default settings
```

```
helix generate "Write a Go function to reverse a string"
```

```
# Generate with specific model and settings
```

```
helix generate "Create a REST API in Python" --model=gpt-4 --max-tokens=2000 --temperature=0.5
```

```
# Stream the response
```

```
helix generate "Explain quantum computing" --stream
```

helix test

Run HelixCode test suite.

```
helix test [flags]
```

Description: Runs comprehensive test suite including unit, integration, and E2E tests.

helix worker

Manage distributed workers.

```
helix worker [subcommand] [flags]
```

Subcommands: - add <host>: Add a new worker - list: List all workers - status: Show worker status - remove <id>: Remove a worker

Examples:

```
# List all workers
```

```
helix worker list
```

```
# Add a new worker
```

```
helix worker add worker1.example.com --user=helix --key=/path/to/key
```

```
# Show worker status
```

helix worker status

Remove a worker

helix worker remove worker-uuid-123

helix notify

Send notifications.

helix notify [message] [flags]

Flags: - --type: Notification type (info/warning/error/success/alert) (default: "info") - --priority: Notification priority (low/medium/high/critical) (default: "medium")

Examples:

Send info notification

helix notify "System started successfully"

Send error notification

helix notify "Database connection failed" --type=error --
priority=high

Send alert

helix notify "High CPU usage detected" --type=alert --
priority=critical

Configuration Management CLI

helix-config

Comprehensive configuration management CLI for HelixCode.

helix-config [command] [flags]

Global Flags

Flag	Type	Default	Description
-c, --config	string	""	Configuration file path
-f, --format	string	"auto"	Configuration format (json, yaml, toml)
-o, --output	string	"json"	Output format (json, yaml, table, pretty)
--session-id	string	""	Session ID for tracking
--user	string	""	User name for audit

Flag	Type	Default	Description
-v, --verbose	bool	false	Verbose output
--dry-run	bool	false	Dry run without making changes
-q, --quiet	bool	false	Quiet mode (no output)
--no-color	bool	false	Disable color output
-i, --interactive	bool	false	Interactive mode
-F, --force	bool	false	Force operation without confirmation
--backup	bool	true	Create backup before making changes
--timeout	duration	30s	Operation timeout
--max-retries	int	3	Maximum number of retries
--show-secrets	bool	false	Show sensitive configuration values
--no-validate	bool	false	Skip configuration validation
--strict	bool	false	Enable strict validation mode
--pretty	bool	true	Pretty print JSON output
--sort-keys	bool	true	Sort object keys in output

Commands

helix-config show

Display current configuration.

```
helix-config show [flags]
```

Flags: - --section: Show specific configuration section - --filter: Filter configuration keys - --depth: Maximum display depth

Examples:

```
# Show all configuration
```

```
helix-config show
```

```
# Show server configuration
```

```
helix-config show --section=server
```

```
# Show database configuration with filter  
helix-config show --section=database --filter="host|port"
```

```
# Show in YAML format  
helix-config show -o yaml
```

helix-config get

Get configuration value.

```
helix-config get <key> [flags]
```

Examples:

```
# Get server port  
helix-config get server.port
```

```
# Get database host  
helix-config get database.host
```

```
# Get LLM provider API key (with secrets shown)  
helix-config get llm.providers.openai.api_key --show-secrets
```

helix-config set

Set configuration value.

```
helix-config set <key> <value> [flags]
```

Examples:

```
# Set server port  
helix-config set server.port 9090
```

```
# Set database host  
helix-config set database.host postgres.example.com
```

```
# Set LLM API key  
helix-config set llm.providers.openai.api_key sk-your-api-key-here
```

```
# Dry run  
helix-config set server.port 9090 --dry-run
```

helix-config delete

Delete configuration key.

```
helix-config delete <key> [flags]
```

Examples:

Delete a configuration key

```
helix-config delete custom.setting
```

Force delete without confirmation

```
helix-config delete custom.setting --force
```

helix-config validate

Validate configuration.

```
helix-config validate [flags]
```

Flags: - --schema: Schema file for validation - --strict: Enable strict validation

Examples:

Validate current configuration

```
helix-config validate
```

Validate with custom schema

```
helix-config validate --schema=custom-schema.json
```

Strict validation

```
helix-config validate --strict
```

helix-config export

Export configuration.

```
helix-config export [filename] [flags]
```

Examples:

Export to stdout

```
helix-config export
```

Export to file

```
helix-config export config-backup.yaml -o yaml
```

Export specific section

```
helix-config export server-config.json --section=server
```

helix-config import

Import configuration.

```
helix-config import <filename> [flags]
```

Flags: - -F, --force: Force import even with validation errors - --conflict: Conflict resolution (first, second, error)

Examples:

Import configuration

```
helix-config import new-config.yaml
```

Force import with conflicts resolved to 'first'

```
helix-config import new-config.yaml --conflict=first
```

Dry run import

```
helix-config import new-config.yaml --dry-run
```

helix-config backup

Create configuration backup.

```
helix-config backup [filename] [flags]
```

Examples:

Create timestamped backup

```
helix-config backup
```

Create named backup

```
helix-config backup my-backup-2024.yaml
```

helix-config restore

Restore configuration from backup.

```
helix-config restore <filename> [flags]
```

Examples:

Restore from backup

```
helix-config restore config-backup-2024-01-20.yaml
```

Dry run restore

```
helix-config restore config-backup.yaml --dry-run
```

helix-config reset

Reset configuration to defaults.

```
helix-config reset [flags]
```

Examples:

Reset all configuration

```
helix-config reset
```

Reset specific section

```
helix-config reset --section=database
```

helix-config reload

Reload configuration from disk.

```
helix-config reload [flags]
```

Examples:

Reload configuration

```
helix-config reload
```

Reload with validation

```
helix-config reload --validate
```

helix-config watch

Watch configuration changes.

```
helix-config watch [flags]
```

Flags: - --interval: Watch interval (default: 5s) - --command: Command to run on changes

Examples:

Watch for configuration changes

```
helix-config watch
```

Watch and run command on changes

```
helix-config watch --command="systemctl restart helixcode"
```

Watch with custom interval

```
helix-config watch --interval=10s
```

helix-config migrate

Migrate configuration between versions.

```
helix-config migrate [flags]
```

Flags: - --from: Source version - --to: Target version - --backup: Create backup before migration

Examples:

Migrate to latest version

helix-config migrate

Migrate from specific version

helix-config migrate --from=1.0.0 --to=1.1.0

helix-config benchmark

Benchmark configuration operations.

helix-config benchmark [flags]

Examples:

Run configuration benchmarks

helix-config benchmark

Benchmark with verbose output

helix-config benchmark --verbose

helix-config template

Manage configuration templates.

helix-config template [subcommand] [flags]

Subcommands: - list: List available templates - apply <template>:
Apply configuration template - create <name>: Create new template
- delete <name>: Delete template

Examples:

List templates

helix-config template list

Apply production template

helix-config template apply production

Create custom template

helix-config template create my-template

helix-config history

View configuration change history.

helix-config history [flags]

Flags: - --limit: Maximum number of entries (default: 50) - --
since: Show changes since date - --user: Filter by user

Examples:

Show recent changes

```
helix-config history
```

Show changes by specific user

```
helix-config history --user=admin
```

Show changes since date

```
helix-config history --since=2024-01-01
```

helix-config schema

Manage configuration schemas.

```
helix-config schema [subcommand] [flags]
```

Subcommands: - show: Show current schema - validate: Validate against schema - update: Update schema

Examples:

Show current schema

```
helix-config schema show
```

Validate configuration against schema

```
helix-config schema validate
```

Update schema

```
helix-config schema update
```

helix-config completion

Generate shell completion scripts.

```
helix-config completion [shell]
```

Supported shells: bash, zsh, fish, powershell

Examples:

Generate bash completion

```
helix-config completion bash > /etc/bash_completion.d/helix-config
```

Generate zsh completion

```
helix-config completion zsh > /usr/local/share/zsh/site-functions/_helix-config
```

helix-config version

Show version information.

```
helix-config version
```

Standalone CLI Tools

Performance Optimization CLI

```
./bin/helixcode-performance-optimization [flags]
```

Flags: - --config: Configuration file path - --baseline: Create performance baseline - --optimize: Run optimization - --report: Generate report - --continuous: Continuous monitoring mode

Examples:

```
# Create performance baseline
```

```
./bin/helixcode-performance-optimization --baseline
```

```
# Run optimization
```

```
./bin/helixcode-performance-optimization --optimize
```

```
# Generate performance report
```

```
./bin/helixcode-performance-optimization --report
```

Security Fix CLI

```
./bin/helixcode-security-fix [flags]
```

Flags: - --scan: Run security scan - --fix: Apply security fixes - --report: Generate security report - --auto: Automatic mode

Examples:

```
# Run security scan
```

```
./bin/helixcode-security-fix --scan
```

```
# Apply automatic fixes
```

```
./bin/helixcode-security-fix --fix --auto
```

```
# Generate security report
```

```
./bin/helixcode-security-fix --report
```

Local LLM CLI

```
./bin/helixcode-local-llm [command] [flags]
```

Commands: - discover: Discover available local models - recommend: Get model recommendations - analytics: Show usage analytics - report: Generate performance report - insights: Get usage insights

Examples:

Discover local models

```
./bin/helixcode-local-llm discover
```

Get model recommendations

```
./bin/helixcode-local-llm recommend --use-case=code-generation
```

Show analytics

```
./bin/helixcode-local-llm analytics
```

Generate insights report

```
./bin/helixcode-local-llm insights --period=7d
```

Docker CLI Commands

Docker Compose Commands

Start all services

```
docker-compose -f docker/docker-compose.yml up -d
```

Start with security scanning

```
docker-compose -f scripts/security_scan/docker-  
compose.security.yml up -d
```

View service status

```
docker-compose ps
```

View logs

```
docker-compose logs -f helixcode
```

Stop all services

```
docker-compose down
```

Rebuild and restart

```
docker-compose up -d --build
```

Security / performance / config tooling

These tools are NOT distributed as the helixcode-* Docker images previously listed here (no such images exist; §11.4.99 correction 2026-05-29) and direct docker runs are not a supported workflow (Rule 4). Use the real entry points:

```
# Run security scanners (root Makefile)
make scan-all          # all scanners; or scan-gosec / scan-
                        trivy / scan-secrets

# Run performance optimization (inner Go cmd)
cd helix_code && go run ./cmd/performance_optimization

# Run configuration validation (inner Go cmd)
cd helix_code && go run ./cmd/config_test
```

Script Commands

Test Scripts

```
# Run all tests
./scripts/run-tests.sh

# Run unit tests only
./scripts/run-tests.sh --unit

# Run with coverage
./scripts/run-tests.sh --coverage

# Run integration tests
./scripts/run-tests.sh --integration
```

Build Scripts

```
# Build all components
make build

# Build with assets
make build-assets

# Build for production
make prod

# Build mobile apps
make mobile
```

Utility Scripts

```
# Setup development environment
./scripts/setup-dev.sh

# Clean build artifacts
./scripts/clean.sh

# Generate documentation
```

```
./scripts/generate-docs.sh
```

```
# Run linting
```

```
./scripts/lint.sh
```

Configuration Examples

Basic Configuration

```
# config/config.yaml
```

```
server:
```

```
  address: "0.0.0.0"
```

```
  port: 8080
```

```
database:
```

```
  host: "localhost"
```

```
  port: 5432
```

```
  user: "helix"
```

```
  dbname: "helixcode_prod"
```

```
redis:
```

```
  host: "localhost"
```

```
  port: 6379
```

```
llm:
```

```
  default_provider: "local"
```

```
  providers:
```

```
    local:
```

```
      enabled: true
```

```
    openai:
```

```
      api_key: "${OPENAI_API_KEY}"
```

Production Configuration

```
# Production configuration
```

```
server:
```

```
  address: "0.0.0.0"
```

```
  port: 8080
```

```
  tls:
```

```
    enabled: true
```

```
    cert_file: "/etc/ssl/certs/helixcode.crt"
```

```
    key_file: "/etc/ssl/private/helixcode.key"
```

```
database:
```

```
  host: "postgres"
```

```
  port: 5432
```

```
  user: "helix"
```

```
  password: "${HELIX_DATABASE_PASSWORD}"
```

```
  sslmode: "require"
```

```
max_connections: 50

redis:
  host: "redis"
  port: 6379
  password: "${HELIX_REDIS_PASSWORD}"

performance:
  cpu_optimization: true
  memory_optimization: true
  garbage_collection: true
  target_throughput: 10000
  target_latency: "50ms"

security:
  scanning:
    enabled: true
    zero_tolerance: true
  headers:
    enabled: true
    csp: "default-src 'self'"
    hsts: true
```

Development Configuration

```
# Development configuration
server:
  address: "127.0.0.1"
  port: 8080

database:
  host: "" # Disabled for testing

redis:
  host: "" # Disabled for testing

logging:
  level: "debug"
  format: "text"

performance:
  cpu_optimization: false
  memory_optimization: false
  garbage_collection: false
```

Exit Codes

Code	Meaning
0	Success

Code	Meaning
1	General error
2	Configuration error
3	Network error
4	Authentication error
5	Permission denied
6	Resource not found
7	Validation error
8	Timeout error
9	Security violation

Environment Variables

Required Variables

Authentication

HELIX_AUTH_JWT_SECRET=your-super-secure-jwt-secret

Database

HELIX_DATABASE_PASSWORD=your-secure-database-password

Redis

HELIX_REDIS_PASSWORD=your-secure-redis-password

Optional Variables

LLM Providers

OPENAI_API_KEY=sk-your-openai-key

ANTHROPIC_API_KEY=sk-ant-your-anthropic-key

GEMINI_API_KEY=your-gemini-key

Notifications

HELIX_SLACK_WEBHOOK_URL=https://hooks.slack.com/...

HELIX_EMAIL_SMTP_SERVER=smtp.gmail.com

Security

HELIX_SECURITY_SCAN_INTERVAL=24h

HELIX_SECURITY_ZERO_TOLERANCE=true

Performance

HELIX_PERFORMANCE_OPTIMIZATION_INTERVAL=1h

HELIX_PERFORMANCE_TARGET_THROUGHPUT=10000

Command Chaining

Pipeline Usage

```
# Chain configuration commands
helix-config get server.port | xargs helix-config set server.port

# Use with other tools
helix-config export | jq '.server.port' | xargs echo "Port is:"
```

Scripting Examples

```
#!/bin/bash
# Backup and update configuration
helix-config backup pre-update-$(date +%Y%m%d)
helix-config set server.port 9090
helix-config validate
if [ $? -eq 0 ]; then
    echo "Configuration updated successfully"
    helix-config reload
else
    echo "Configuration validation failed"
    exit 1
fi
```

Troubleshooting CLI Issues

Common Problems

Command Not Found

```
# Check if binary exists
ls -la bin/helixcode

# Add to PATH
export PATH=$PATH:/path/to/helixcode/bin

# Check permissions
chmod +x bin/helixcode
```

Configuration Not Loaded

```
# Check configuration file
cat config/config.yaml

# Validate syntax
yamllint config/config.yaml
```

```
# Check file permissions
ls -la config/config.yaml
```

Connection Refused

```
# Check if server is running
curl http://localhost:8080/health
```

```
# Check port binding
netstat -tlnp | grep 8080
```

```
# Check firewall
iptables -L | grep 8080
```

Permission Denied

```
# Check user permissions
id
```

```
# Check file permissions
ls -la /etc/helixcode/
```

```
# Run with sudo if necessary
sudo helixcode server
```

Debug Mode

```
# Enable debug logging
helixcode --debug server
```

```
# Verbose configuration output
helix-config --verbose show
```

```
# Dry run mode
helix-config set server.port 9090 --dry-run
```

Log Analysis

```
# View recent logs
tail -f /var/log/helixcode/server.log
```

```
# Search for errors
grep "ERROR" /var/log/helixcode/*.log
```

```
# Count error types
grep "ERROR" /var/log/helixcode/*.log | cut -d' ' -f4 | sort |
    uniq -c
```

CodeGraph integration (MCP code-graph server)

HelixCode incorporates **CodeGraph** (@colbymchenry/codegraph, MIT) as a pinned, vendored third-party tool under tools/codegraph/. CodeGraph parses the codebase into a local SQLite knowledge graph of symbols and edges and serves it to CLI coding agents over the **Model Context Protocol (MCP)**, so an agent queries the graph instantly instead of running grep/glob/Read scans.

It is not part of the helix binary — it is a developer-tooling layer wired into five CLI agents (Claude Code, OpenCode, Kimi CLI, Crush, Qwen Code).

Install + initialize + scan

```
# Install the pinned CodeGraph runtime (idempotent).
tools/codegraph/install.sh

# Initialize + scan (re-runnable any time).
tools/codegraph/node_modules/.bin/codegraph init -i          # repo
root
cd helix_code && ../tools/codegraph/node_modules/.bin/codegraph
init -i

# Inspect the graph – non-zero node/edge/file counts prove the
scan worked.
tools/codegraph/node_modules/.bin/codegraph status .
tools/codegraph/node_modules/.bin/codegraph query Provider
```

Per-agent registration

```
tools/codegraph/agents/register-claude.sh    # .mcp.json +
mcp__codegraph__* permission
tools/codegraph/agents/register-opencode.sh  # opencode.jsonc
tools/codegraph/agents/register-kimi.sh      # ~/.kimi/mcp.json
tools/codegraph/agents/register-crush.sh     # .crush.json
tools/codegraph/agents/register-qwen.sh      # .qwen/settings.json
```

Each agent then exposes eight codegraph_* MCP tools (codegraph_search, codegraph_context, codegraph_callers, codegraph_callees, codegraph_impact, codegraph_node, codegraph_files, codegraph_status).

Full detail: tools/codegraph/README.md and the incorporation plan at docs/research/codegraph/incorporation-plan.md.

This CLI reference provides comprehensive coverage of all HelixCode command-line tools and their usage patterns. For additional help, use `--help` flag with any command or refer to the specific documentation sections.

Sources verified 2026-05-29: <https://go.dev/dl/> , <https://www.postgresql.org/support/versioning/> , <https://github.com/redis/redis/releases> , <https://platform.claude.com/docs/en/docs/about-claude/models/overview>

Verified against latest official sources on 2026-05-29. Go go1.26.3 latest (go.dev/dl) per CLAUDE.md §3.1. PostgreSQL 15 still supported (latest 15.18); Redis latest stable 8.8.0 (7+ pin still valid). The docker-compose command examples here are illustrative of the underlying stack only; per CONST/Rule 4 + §11.4.76 the supported workflow is the `./helix facade (podman host) / the containers submodule`, not hand-run docker — the doc already carries the §11.4.99 correction note that the `helixcode-* Docker` images do not exist.

Negative findings: example model IDs (`gpt-4`, `llama-3-8b`) are illustrative; per CONST-036 the authoritative model list is LLMsVerifier-sourced at runtime — run `helixcode llm models list`. `claude-3-*` / `gpt-3.5-turbo` style IDs are legacy per Anthropic's current model overview (Claude 3 family retired) and not re-pinned here to avoid guessing. OpenAI docs returned HTTP 403 to automated fetch.